Introduktion till C++



Uppgift 8 – Timer

Uppgiftens målsättning

Den studerande ska få en förståelse för hur man skapar en högprecisionstimer samt att timern kan användas i spelprojekt senare. Uppgiften ger även en introduktion till att skapa egna bibliotek som kan inkluderas i projekt.

Uppgift

Vi ska skriva en klass för en högprecisionstimer. Gränssnittet för klassen ska ligga i `namespace CommonUtilities` och se ut på följande sätt:

```
namespace CommonUtilities
{
class Timer
{
public:
    Timer(float aFixedTickTime = 1.f/60.f);
    Timer(const Timer &aTimer) = delete;
    Timer& operator=(const Timer &aTimer) = delete;

    void Update();

    float GetDeltaTime() const;
    double GetTotalTime() const;

    int GetFixedRateTickDeltaCount() const;
    uint64_t GetFixedRateTickCount() const:

};
}
```

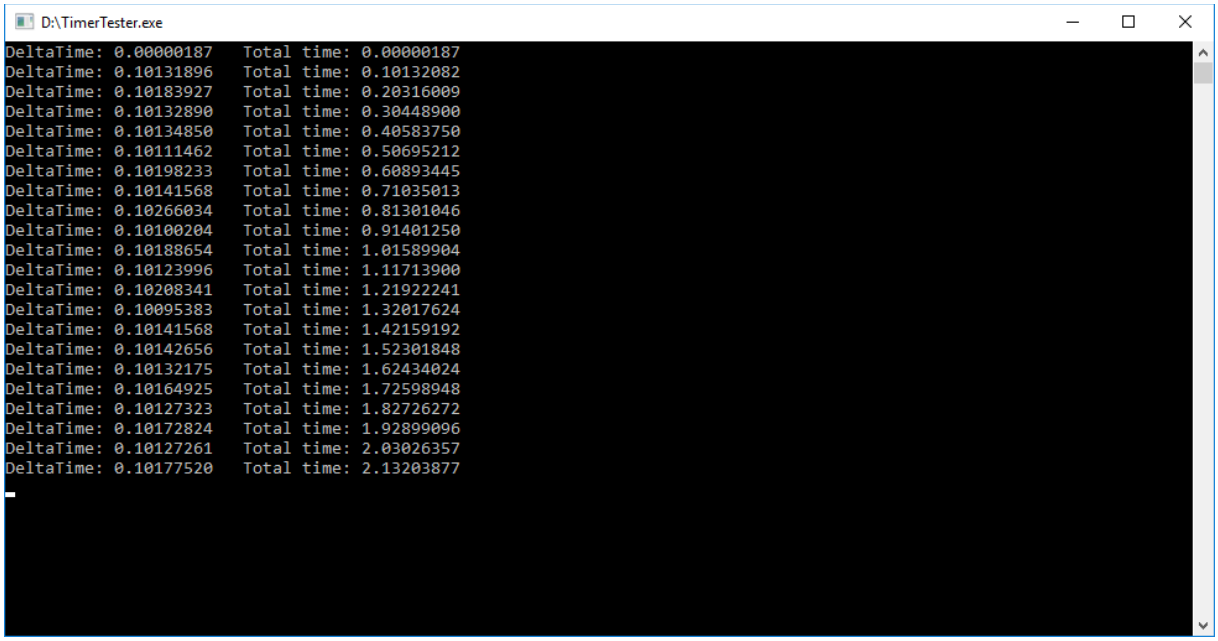
Kom ihåg att gränssnittet för en klass bara innehåller de publika medlemmarna – det innebär att vi är fria att lägga till alla privata medlemmar vi känner att vi behöver.

För att få reda på tiden ska `std::chrono::high_resolution_clock` användas. Det är upp till var och en att använda Google och andra resurser för att ta

reda på hur den fungera. Ett typiskt projekt innehåller exakt en sådan här timer.

Vi ska skapa timern som ett statiskt bibliotek som kan kallas *Common Utilities*.

Gör ett program som inkluderar det statiska biblioteket för att få tillgång till timern. Denna ska uppdateras ca 10 gånger i sekunden i oändlighet. För varje uppdatering (dvs. varje ”*frame*”) ska tiden skrivas ut i sekunder med flera decimaler (både *delta time* och total tid). Använd Windows-funktionen Sleep för att vänta mellan varje Update(). Se exempel nedan:



Viktigt: Både .h/.cpp filen ska lämnas in.

Features

`Update()` ska köras en gång varje frame. Det är här klassen frågar systemet om tiden, så vi vill bara anropa denna funktion en gång per frame så att vi har samma värde för tiden under en hel frame. Detta är typiskt det första som händer varje frame.

`GetDeltaTime()` ger tiden i sekunder för hur lång tid föregående frame tog, dvs. skillnaden mellan den tid vi hade föregående frame och den tid vi har just nu. Om vi har 60 fps innebär det att vi hinner 60 frames under en sekund och att varje frame tar 1 / 60 = 0,0167s eller 16,7ms. Om timern är helt ny är *delta time* 0. Det är detta tal vi multiplicerar all förflyttning med så att vi får en framerate-oberoende förflyttning. Ska ge samma resultat som att anrop `GetTotalTime()` förra framen och denna frame och ta skillnanden

`GetTotalTime()` ger antalet sekunder sedan timern skapades. För en helt ny timer är detta värde 0. Detta returneras som en double då det ger oss en bättre precision än float. Med en double kan spelet köra extremt länge innan vi får dålig precision. Denna funktion är sällan intressant, men kan vara bra att ha.

`GetFixedRateTickDeltaCount()` ger antalet fixed rate uppdateringar som ska ske under denna frame, baserat på tiden som gått mellan förra och förrförra uppdateringen. Ska ge samma resultat som att anropa `GetFixedRateTickCount()` förra framen och denna frame och ta skillnanden

`GetFixedRateTickCount()` ger antalet fixed rate uppdateringar som skett sen timerns startades. Implementera genom att ta tiden i sekunder sen timern skapades, dela på "fixed tick time" och avrunda till en uint64_t. Avrundingen görs på valfritt sätt. Valet kommer styra när ticks sker, men spelar ingen större roll).

Inlämning

För denna inlämning ska både en Debug- och en Release-version lämnas in i *Exe*-mappen så att det är tydligt att ni länkat in båda versionerna korrekt.

Tänk också på att ni måste lämna in koden för båda projekten!